

Assignment 1 : Supervised Machine Learning

Renjiro Owen
Victoria University Wellington
Wellington, New Zealand
owenrenj@myvuw.ac.nz

August 19, 2026

1 Decision Tree Method

1.1 Results

The decision tree algorithm was trained using the rtg A, rtg B, and rtg C training dataset. The generated decision tree was constructed using entropy-based Information Gain, with a stopping threshold of $IG < 0.00001$. The overall training classification accuracy achieved was:

$$Accuracy_{rtgA} = 100\% \tag{1}$$

$$Accuracy_{rtgB} = 100\% \tag{2}$$

$$Accuracy_{rtgC} = 100\% \tag{3}$$

The classification accuracy was calculated using:

$$Accuracy = \frac{\text{Number of correctly classified instances}}{\text{Total number of instances}} \tag{4}$$

1.2 Minimal Entropy

For each dataset, the feature resulting in the minimum entropy was identified by calculating the weighted entropy after splitting the entire training dataset on each available feature. The feature with the lowest resulting entropy represents the split that produces the purest separation of class labels and therefore provides the greatest reduction in uncertainty.

For *rtg A*, Feature 0 produced the minimum entropy value of 0.285506 with an Information Gain of 0.116673. Therefore, Feature 0 was selected as the root splitting feature of the generated decision tree. For *rtg B*, Feature 3 resulted in the lowest entropy value of 0.900495 with an Information Gain of 0.053199, making it the most effective feature for the initial split. For *rtg C*, Feature 3 achieved an entropy value of 0.000000 and an Information Gain of 0.904381, indicating that this feature completely separates the classes in the training dataset.

These observations were made by comparing the entropy values obtained from all features. The feature with the smallest entropy (or equivalently the highest Information Gain) was considered the most informative feature and was chosen by the decision tree algorithm for splitting the dataset.

1.3 Unused Feature

The features utilized by the decision tree were different for each dataset. For *rtg A*, Feature 3 was not used in constructing the decision tree. This occurred because the other features provided sufficient Information Gain to separate the classes, allowing the tree to reach pure leaf nodes before Feature 3 was considered. Since the stopping criteria were satisfied once the nodes became sufficiently pure, Feature 3 did not contribute additional useful information for classification.

For *rtg B*, all available features were utilized in the construction of the decision tree. This indicates that each feature contributed useful information during the splitting process. The tree required all features to further separate the data until the stopping criteria were reached.

For *rtg C*, only Feature 3 was used in constructing the decision tree, while the remaining features were excluded. This was because Feature 3 provided a very strong separation of the classes, achieving an entropy of 0 and a high Information Gain of 0.904381. After splitting on Feature 3, the resulting nodes were already pure, meaning additional features were unnecessary. Therefore, the remaining features were not selected because they did not provide any further improvement to the classification.

Overall, features were excluded from the tree-building process when they did not provide enough additional Information Gain or when previous splits had already produced pure leaf nodes. The decision tree only selects features that reduce class uncertainty and ignores redundant or unnecessary attributes.

1.4 Rtg C Discussion

1.4.1 Generalisation Ability of the Generated Tree

The generated decision tree is not expected to perform well on unseen data. The tree structure indicates that Feature 3 alone is able to completely separate the training instances, resulting in a very simple tree where the split is entirely dependent on a single feature. Although this produces perfect classification performance on the training dataset, it may not generalise well to unseen data because the model relies heavily on Feature 3.

If new instances contain feature values that were not represented in the training dataset, the tree may not have sufficient information from other features to make a reliable prediction. Therefore, the tree structure suggests a risk of poor generalisation due to its dependency on a single feature rather than learning patterns from multiple attributes.

1.4.2 Unused Features in Tree Construction

For the rtg C dataset, only Feature 3 was utilized in constructing the decision tree, while the remaining features were not selected. This occurred because Feature 3 provided a very strong separation between the two classes, achieving an entropy of 0 and a high Information Gain of 0.904381.

Since splitting on Feature 3 resulted in pure leaf nodes, there was no need for the decision tree to consider additional features. The other attributes may contain useful information; however, their contribution was unnecessary after Feature 3 had already completely separated the training data. Therefore, these features were excluded because they did not provide additional Information Gain during the tree-building process.

1.4.3 Potential Preprocessing Improvements

Although a test dataset is not available, several preprocessing approaches could be considered to improve the generalisation ability of the generated tree. Firstly, Feature 3 could be investigated further because the tree's complete dependency on this single attribute may indicate that the dataset contains a dominant or potentially artificial feature. Removing Feature 3 before training could encourage the model to learn from other attributes and produce a more generalised decision boundary.

Additionally, feature selection techniques could be applied to identify and retain informative attributes while reducing dependency on a single feature. Tree pruning or increasing the Information Gain stopping threshold could

also be considered to prevent the tree from creating overly specific splits based only on the training data.

These preprocessing steps may reduce overfitting and improve the ability of the decision tree to classify unseen instances.

2 Exploring Classification Algorithms

2.1 Missing Data Processing and Data Normalization

For the missing-data handling experiment, two imputation strategies, mean imputation and median imputation, were applied to the dataset before training the Decision Tree classifier. The target variable was set as diagnosis, and the dataset was split into training and testing sets using a 70:30 ratio with a fixed random state of 42 to ensure reproducibility. A Decision Tree classifier with a single plausible hyperparameter configuration (maximum depth = 8) was used to evaluate the effect of each imputation method. The results showed that both mean and median imputation achieved the same classification accuracy of 91.23%, indicating that the choice of imputation strategy had no significant impact on the model performance for this dataset. Since mean imputation achieved the highest accuracy (91.23%), it was selected as the best missing-data handling method. The completed dataset generated using the selected imputation approach was then used for all subsequent tasks.

Imputation Method	Accuracy (%)
Mean Imputation	91.23
Median Imputation	91.23

Table 1: Decision Tree Accuracy Comparison Using Different Missing-Data Imputation Strategies

Standardization and Min-Max scaling were applied to the completed dataset to investigate the impact of feature normalization on KNN and Decision Tree classifiers. The dataset was evaluated using a fixed train-test split with one plausible hyperparameter configuration for each model: $K = 5$ for KNN and maximum depth = 8 for the Decision Tree. The results are presented in Table 2. Without normalization, the KNN classifier achieved an accuracy of only 69.59%, while applying standardization and Min-Max scaling significantly improved its performance to 96.49% and 97.08%, respectively. This demonstrates that KNN is highly affected by feature scaling because it relies on distance calculations between samples. In contrast, the

Decision Tree classifier achieved the same accuracy of 91.23% across all three datasets, indicating that normalization had no noticeable effect on its performance. This is expected because Decision Trees make decisions based on feature thresholds rather than distance measurements. Therefore, normalization techniques have a much greater impact on KNN compared with Decision Tree models, with Min-Max scaling providing the best performance improvement for KNN in this experiment.

Normalization Method	KNN (%)	Decision Tree (%)
No Normalization	69.59	91.23
Standardization	96.49	91.23
Min-Max Scaling	97.08	91.23

Table 2: Accuracy Comparison of KNN and Decision Tree Models with Different Normalization Techniques

2.2 Classifier Exploration and Hyperparameter Tuning

The performance of four classification algorithms, KNN, Decision Tree, AdaBoost, and Random Forest, was evaluated by varying their key hyperparameters while keeping the dataset split consistent. The results are shown in Table 3. For KNN, changing the number of neighbors had a noticeable impact on performance. The best result was achieved with `n_neighbors = 9`, producing the highest accuracy of 97.66% and an F1 score of 0.9677. Smaller values of k can make the model more sensitive to noise, while larger values may oversmooth the decision boundary and reduce classification performance.

For the Decision Tree classifier, increasing the maximum tree depth from 2 to 8 improved accuracy from 90.64% to 91.23%. However, further increasing the depth to 14 did not provide additional improvement, suggesting that the model had already captured the important patterns in the dataset and deeper trees did not contribute to better generalization. The similar precision, recall, and F1 scores between depth values of 8 and 14 indicate that additional complexity did not improve predictive performance.

AdaBoost showed continuous improvement as the number of weak learners increased. Increasing `n_estimators` from 10 to 30 improved accuracy from 94.74% to 96.49%, with corresponding improvements in recall and F1 score. This occurs because boosting combines multiple weak learners to reduce prediction errors and create a stronger ensemble model.

Random Forest also benefited from increasing the number of trees. The best performance was achieved with `n_estimators = 50`, reaching an accuracy of 97.08%, precision of 1.0000, recall of 0.9219, and an F1 score of 0.9594. However, increasing the number of trees further to 60 slightly reduced performance, indicating that additional estimators provided limited benefit.

Overall, KNN with `n_neighbors = 9` achieved the highest accuracy of 97.66%, making it the best-performing classifier in this experiment. Random Forest with 50 trees also achieved strong performance, demonstrating the effectiveness of ensemble learning methods. The superior performance of KNN is likely due to the normalized feature values and the dataset’s clear separation between classes, allowing distance-based classification to identify similar samples effectively. Random Forest performed well because combining multiple decision trees reduces overfitting and improves model stability compared with a single Decision Tree. AdaBoost also achieved competitive results by iteratively improving weak learners, while the standalone Decision Tree showed lower performance due to its higher tendency to overfit and sensitivity to the selected tree depth.

Classifier	Hyperparameter	Value	Accuracy (%)	Precision	Recall	F1 Score
KNN	n_neighbors	3	97.08	1.0000	0.9219	0.9594
		9	97.66	1.0000	0.9375	0.9677
		15	95.91	1.0000	0.8906	0.9421
		21	96.49	1.0000	0.9063	0.9508
Tree	max_depth	2	90.64	0.9444	0.7969	0.8644
		8	91.23	0.9298	0.8281	0.8760
		14	91.23	0.9298	0.8281	0.8760
AdaBoost	n_estimators	10	94.74	0.9825	0.8750	0.9256
		20	95.91	0.9831	0.9063	0.9431
		30	96.49	0.9833	0.9219	0.9516
Forest	n_estimators	10	95.32	0.9828	0.8906	0.9344
		30	95.91	0.9831	0.9063	0.9431
		50	97.08	1.0000	0.9219	0.9594
		60	96.49	1.0000	0.9063	0.9508

Table 3: Classifier Performance with Different Hyperparameter Configurations

2.3 Feature Engineering and Dimensionality Reduction

2.3.1 Feature Selection using Pearson Correlation

Pearson correlation was applied to identify the relationship between each feature and the target variable (diagnosis). A correlation threshold of 0.6 was

selected, where only features with an absolute correlation coefficient greater than 0.6 were retained. Features satisfying this condition were considered highly relevant for classification. The Decision Tree classifier was then trained using the reduced feature set, and its performance was compared with the model trained using all original features.

The correlation coefficients between the features and the diagnosis variable are shown in Table 4. Based on the threshold value of 0.6, ten features were selected: `radius_mean`, `perimeter_mean`, `area_mean`, `concavity_mean`, `concave points_mean`, `radius_worst`, `perimeter_worst`, `area_worst`, `concavity_worst`, and `concave points_worst`. These selected features represent measurements that have the strongest relationship with the diagnosis outcome.

The feature selection process reduced the number of input features from 31 to 10. The classification accuracy before and after feature selection is presented in Table 5. The reduced feature set slightly improved the Decision Tree performance from 90.06% to 90.64%, indicating that removing weakly correlated features can improve model generalisation.

2.3.2 Feature Extraction using PCA

Principal Component Analysis (PCA) was implemented using the scikit-learn library to reduce the dimensionality of the dataset. Since PCA is sensitive to feature scale, the data was normalised before applying PCA to ensure that features with larger numerical ranges did not dominate the principal components. The same transformation learned from the training data was applied to the testing data to maintain consistency.

The explained variance ratio of each principal component is shown in Table 6. The first principal component explains 49.81% of the variance, while the first ten components together preserve 94.90% of the total variance. This indicates that PCA can significantly reduce dimensionality while retaining most of the important information from the original dataset.

The Decision Tree classifier was evaluated using a fixed hyperparameter setting with a maximum tree depth of 4. The comparison between the original feature space, Pearson correlation feature selection, and PCA based feature extraction is shown in Table 7. PCA achieved the highest accuracy of 92.40%, demonstrating that transforming the original features into principal components can improve classification performance.

Overall, Pearson correlation feature selection provided a small improvement by removing less relevant features, while PCA achieved a larger improvement by creating new feature combinations that capture the most significant variance in the dataset. The results suggest that PCA was more effective for this dataset because it reduced noise and improved the Decision

Feature	Correlation Coefficient
concave points_worst	0.7908
perimeter_worst	0.7829
radius_worst	0.7765
concave points_mean	0.7683
perimeter_mean	0.7426
area_worst	0.7338
radius_mean	0.7300
area_mean	0.7090
concavity_mean	0.6863
concavity_worst	0.6492
compactness_mean	0.5965
compactness_worst	0.5910
radius_se	0.5671
perimeter_se	0.5561
area_se	0.5482
texture_worst	0.4569
smoothness_worst	0.4215
symmetry_worst	0.4163
texture_mean	0.4152
concave points_se	0.3907
smoothness_mean	0.3586
symmetry_mean	0.3305
fractal dimension_worst	0.3239
compactness_se	0.2930
concavity_se	0.2378
fractal dimension_se	0.0780
id	0.0398
symmetry_se	-0.0065
texture_se	-0.0083
fractal dimension_mean	-0.0128
smoothness_se	-0.0670

Table 4: Pearson Correlation Coefficients between Features and Diagnosis

Feature Set	Accuracy
Original Features	90.06%
Pearson Correlation Feature Selection	90.64%

Table 5: Decision Tree Accuracy Before and After Pearson Feature Selection

Principal Component	Explained Variance Ratio
PC1	0.4981
PC2	0.1740
PC3	0.0691
PC4	0.0661
PC5	0.0395
PC6	0.0341
PC7	0.0294
PC8	0.0169
PC9	0.0115
PC10	0.0103

Table 6: Explained Variance Ratio of PCA Components

Method	Accuracy
Before Feature Selection	90.06%
Pearson Correlation Selection	90.64%
PCA Feature Extraction	92.40%

Table 7: Accuracy Comparison of Feature Reduction Methods

Tree’s ability to separate the two diagnosis classes.

2.4 Comparison between PCA Feature Extraction and Feature Selection

To provide a fair comparison between PCA-based feature extraction and Pearson correlation-based feature selection, the number of PCA components was set to match the number of features retained after feature selection. Pearson correlation feature selection reduced the original dataset to 10 features; therefore, PCA was configured to retain 10 principal components. This ensures that both approaches use the same number of dimensions when training the Decision Tree classifier.

A Decision Tree model was trained separately using the PCA-transformed dataset and the feature-selected dataset. The classification accuracy of both approaches was then compared to evaluate the effectiveness of reducing the feature space. The results are shown in Table 8.

The results show that PCA achieved a higher accuracy than Pearson correlation feature selection, improving the Decision Tree performance from 90.64% to 92.40%. This improvement occurs because PCA creates new features by combining information from the original variables, allowing the

Dimensionality Reduction Method	Decision Tree Accuracy (%)
Pearson Correlation (10 features)	90.64
PCA (10 components)	92.40

Table 8: Comparison between Feature Selection and PCA using the same number of dimensions

model to capture hidden patterns and reduce noise within the dataset.

Feature selection has the advantage of maintaining the original feature meanings, making the selected features easier to interpret. For example, the retained features correspond directly to physical measurements such as radius, perimeter, and concavity, which can provide meaningful insights into the diagnosis prediction. Additionally, feature selection reduces computational complexity by removing irrelevant features without transforming the data. However, feature selection may discard useful information because it evaluates features individually and does not consider relationships between multiple features.

In contrast, PCA considers the relationships between all input variables and creates a smaller set of transformed features that preserve the maximum amount of variance. This can improve classification performance and reduce the effect of correlated features. However, PCA components are combinations of the original features and therefore lack direct interpretability. Furthermore, PCA requires feature normalisation before transformation, adding an additional preprocessing step.

Overall, PCA provided better predictive performance for the Decision Tree classifier, while Pearson correlation feature selection provided better interpretability. The choice between these methods depends on the objective of the application: PCA is more suitable when classification accuracy is the priority, whereas feature selection is preferable when understanding the contribution of individual features is important.

2.5 10-Fold Cross-Validation Evaluation

To evaluate the reliability and stability of the classification models, 10-fold stratified cross-validation was implemented for both K-Nearest Neighbour (KNN) and Decision Tree classifiers. Stratified cross-validation was used to ensure that each fold maintained a similar distribution of the two diagnosis classes. The KNN classifier was evaluated using two hyperparameter settings, $k = 3$ and $k = 9$, while the Decision Tree classifier was evaluated using maximum tree depths of 2 and 8.

For each model configuration, the classification accuracy was recorded for

every fold. The mean accuracy and standard deviation were calculated to measure overall performance and model stability. A lower standard deviation indicates that the model performance is more consistent across different training and validation splits.

The results of the 10-fold cross-validation are presented in Table 9.

Fold	KNN (k=3)	KNN (k=9)	Tree (depth=2)	Tree (depth=8)
Fold 1	98.25	100.00	89.47	89.47
Fold 2	98.25	98.25	85.96	91.23
Fold 3	98.25	96.49	89.47	92.98
Fold 4	94.74	94.74	80.70	87.72
Fold 5	94.74	92.98	91.23	89.47
Fold 6	92.98	92.98	94.74	98.25
Fold 7	100.00	98.25	92.98	91.23
Fold 8	94.74	96.49	96.49	92.98
Fold 9	96.49	98.25	92.98	92.98
Fold 10	98.21	98.21	94.64	92.86
Mean	96.66	96.66	90.87	91.92
Std Dev	2.14	2.28	4.49	2.74

Table 9: 10-Fold Cross-Validation Accuracy Results for KNN and Decision Tree Models

The results show that both KNN configurations achieved the highest classification accuracy, with both $k = 3$ and $k = 9$ obtaining a mean accuracy of 96.66%. The similar performance between the two values of k indicates that the choice of neighbourhood size has limited impact on this dataset. However, KNN with $k = 3$ achieved a slightly lower standard deviation (2.14%) compared with $k = 9$ (2.28%), suggesting that the smaller neighbourhood size provides slightly more consistent predictions across different folds.

For the Decision Tree models, increasing the maximum depth from 2 to 8 improved the mean accuracy from 90.87% to 91.92%. This improvement occurs because a deeper tree can capture more complex relationships within the dataset. However, the Decision Tree with maximum depth 2 produced the highest standard deviation of 4.49%, indicating that the shallow tree was more sensitive to changes in the training data. In comparison, the depth-8 Decision Tree achieved a lower standard deviation of 2.74%, demonstrating improved stability.

Overall, KNN demonstrated the highest performance and reliability among the evaluated models, achieving the best accuracy with relatively low variation between folds. The Decision Tree models provided lower accuracy but offer greater interpretability because the decision-making process can be represented through feature-based rules. The results suggest that KNN is more suitable when predictive performance and stability are prioritised, while De-

cision Tree models may be preferred when model interpretability is more important.

3 Statement

I declare that this assignment is my own work. The implementation, experimentation, model training, and evaluation were completed by me. Artificial intelligence Chatgpt [1] were used only for language assistance during the preparation of this report.

References

- [1] OpenAI, “ChatGPT (gpt-5.5),” <https://chatgpt.com/>, 2026, large language model. Accessed: 2026-07-20.